Pattern matching

© 2025 Слинчук Данил.
(редактировал: Косарев Дмитрий)

Creative Commons Attribution 4.0 International

1 Сопоставление с образцом

1.1 Основные подходы

Сопоставление с образцом (pattern matching, PM) — ключевая часть функционального программирования, крайне важно уметь ее компилировать эффективно, так как она занимает существенную долю от всего исполнения программы. Существует два разных способа компилировать сопоставление с образцом:

- Автоматы с возвратами — минимизируют размер кода, что влияет на скорость исполнения, так как более короткие программы исполняются быстрее (статическая стоимость)
- Диаграммы решений — минимизируют количество проверок, но порождают более длинный код (динамическая стоимость)

1.1.1 Определение

Пусть p будет паттерном, а v значением какого-то типа. Значение v подходит под образец p (англ. p matches v) (записывается $p \preccurlyeq v$), если подходит одно из правил ниже

$_ \preccurlyeq v$	
$x \preccurlyeq v$	
$(p_1 \vert p_2) \preccurlyeq v$	т.т.к $p_1 \preccurlyeq v$ или $p_2 \preccurlyeq v$
$(p_1; ...; p_a) \preccurlyeq (v_1; ...; v_a)$	т.т.к. $p_i \preccurlyeq v_i : \ \forall i \in [1 \ldots a]$
$C(p_1; ...; p_a) \preccurlyeq C(v_1; ...; v_a)$	т.т.к. $(p_1; ...; p_a) \preccurlyeq (v_1; ..; v_a)$

1. Первые два правила говорят о том, что под переменную (имя) или wildcard будет подходить любое значение.

2. Третье правило говорит о том, что под or-паттерн будет подходить значение, которое подходит под первый или второй паттерн.

3. Четвертое правило говорит о том, что под n-ку арности a будет подходить значение такой же арности, если все компоненты значения будут подходить под соответствующие паттерны.

4. Пятое правило говорит о том, что под конструктор C с a аргументами будет подходить конструктор с таким же именем и такой же арностью, если его аргументы будут подходить (см. правило 4).

2 Автоматы с возвратами

2.1 Во что будем компилировать?

Далее будет приведены конструкции лямбда-кода, которые будут порождаться при компиляции PM.

- `let (x1 e1) (x2 e2) ... (xn en) e`

  — let необходим для определения имен для подвыражений, который мы будет сопоставлять.

- `catch` l_1 `with` l_2

  Статические исключения exit и ловушки: catch l_1 with l_2. Если при оценке тела l_1 встречается exit, то результатом оценки catch l_1 with l_2 будет результат обработчика l_2, иначе - результат l_1. В отличие от динамических исключений, статические исключения компилируются непосредственно как переходы к соответствующим обработчикам (плюс некоторое окружение), в то время как ловушки не генерируют никакого кода.

- `switch l with`
  `case c1: l1`
  `...`
  `case ck: lk`
  `default: d`

  Результатом конструкции switch является оценка li, соответствующего конструктору ci, являющемуся головным для значения v из l. Если головной конструктор v не встречается в списке case, то результатом будет оценка выражения по умолчанию d.

  Выражение по умолчанию default: d может быть опущено. В этом случае поведение switch для нераспознанных значений будет неопределенным. Мы используем ключевое слово switch*, чтобы выделить конструкции switch без условия по умолчанию.

- `field n x`

  — функция получения n-ого элемента n-ки x

- `l1; l2; ()`

  Последовательное исполнение и юнит.

2.2 Начальное состояние

Пусть у нас есть следующее выражение PM:

match x **with** p^1 -> l_1 | p^1 -> l_2 | ... | p^m -> l_m

Пусть

$$\overrightarrow{x} = (x_1 ... x_n),\ P \to L = \begin{bmatrix} p_1^1 & p_2^1 & \dots & p_n^1 \to l^1 \\ p_1^2 & p_2^2 & \dots & p_n^2 \to l^2 \\ \vdots & \vdots & \ddots & \vdots \\ p_1^m & p_2^m & \dots & p_n^m \to l^m \end{bmatrix}$$

В общем случае x является n-кой из значений $x_1 ... x_n$, а весь pattern matching можно представить в виде матрицы, где стоки будут состоять из n-ок паттернов и соответствующего значения l^i, которое будет результатом в случае

подходящего паттерна. Тогда вся компиляция такого PM будет выглядеть следующим образом:

$$\texttt{catch } C(\overrightarrow{x}, \begin{bmatrix} p^1 & \to & l^1 \\ p^2 & \to & l^2 \\ \vdots & & \vdots \\ p_1^m & \to & l^m \end{bmatrix}) \texttt{ with (failwith "Partial match"),}$$

где C — функция порождающая код для PM.

Рассмотрим вырожденный случай, если $n = 0$:

$$C\left((), \begin{bmatrix} \to & l^1 \\ \to & l^2 \\ & \vdots \\ \to & l^m \end{bmatrix}\right) = l^1$$

Если $n \neq 0$, то возможны 4 варианта:

- В первом столбце все переменные
- В первом столбце все конструкторы
- Только одна строка с or-паттернами
- Все остальные случаи

Рассмотрим каждый из них отдельно

2.2.1 Правило для переменных

Если первый столбец из переменных $(y^1, ..., y^n)$, то

$$C(\overrightarrow{x}, P \to L) = C((x_2 ... x_n), P' \to L'), \text{ где}$$

$$P' \to L' = \begin{bmatrix} p_2^1 & ... & p_n^1 \to \textbf{let } (y^1 x_1)\ l^1 \\ p_2^2 & ... & p_n^2 \to \textbf{let } (y^2 x_1)\ l^2 \\ \vdots & \ddots & \vdots \\ p_2^m & ... & p_n^m \to \textbf{let } (y^m x_1)\ l^m \end{bmatrix}$$

Таким образом мы избавляемся от первого столбца в нашей изначальной матрице, путем того, что именуем x_1 как y^i для соответствующего значения l^i.

Пример:

```
match (x1, x2) with          match x2 with
| (a, p1^2) -> l1     ⟹     | p1^2 -> let a = x1 in l1
| (b, p2^2) -> l2           | p2^2 -> let b = x1 in l2
```

2.2.2 Правило конструкторов

Все паттерны в первом столбце — это конструкторы, они образуют множество C. Мы специализируем матрицу под каждый конструктор: мы можем переместить строки матрицы, если первые конструкторы у них разные, а затем ”разрезать” матрицу по конструкторам, так как если значение подходит под один из них, то оно не подходит ни под какой другой, однако мы не можем менять порядок для одинаковых конструкторов. Формально это выглядит следующим образом. Для каждого конструктора c в C мы определяем специализированную матрицу положений $S(c, P \to L)$ путем отображения следующих преобразований на строки P.

p_1^i	$S(c, P \to L)$
$c(q(i_1, ..., q_b^i)$	$q_1^i, ..., q_a^i, p_2^i, ..., p_n^i$
$c'(q_1^i, ..., q_b^i), c \neq c'$	Нет строки

Также определяем лямбда-выражение $r(c)$

```
r(c) = (let (y1 (field 0 x1))
    ...
    (ya (field (a-1) x1 ))
C((y1, ... , ya; x2, ... , xn); S(c; P -> L)))
```

Пример:

```
 match (x1, x2) with
 | C1 a, ... -> l1
 | C2 (a, b), ... -> l3      ⟹
 | C1 b, ... -> l2

match Obj.(tag (repr x1)) with
| C1 -> match Obj.(field 0 (repr x1)), x2 with
    | a, ... -> l1
    | b, ... -> l2
| C2 -> match Obj.(field 0 (repr x1)), Obj.(field 1 (repr x1)), x2 with
```

```
| a,b, ... -> l3
```

2.2.3 Or-паттерны

Наша матрица — одна строка, где первый столбец полностью состоит из or-паттернов. Мы разбиваем матрицу на две: в первой будет matching x_1 с or-паттернами, во второй - все остальное. Такой подход выигрывает тем, что мы не дублировали код, однако этот способ не позволяет напрямую использовать переменные в or-паттернах, так как в этом случае они будут ”видны” только в первой матрице, где нет возвращаемого значения. В OCaml эта проблема решена путем расширения конструкции switch case, которое мы рассмотрим ниже, поэтому там можно использовать переменные в or-паттернах и в правых частях matching.

$$P = ((q_1|...|q_o)p_2...p_n \to l$$

$$C\left((x_1), \begin{bmatrix} q_1 \to () \\ \vdots \\ q_o \to () \end{bmatrix}\right); C((x_2, ..., x_n), (p_2, ..., p_n \to l))$$

2.2.4 Если ничего не подошло (mixture rule)

Если ничего из предыдущего не подходит, то матрица $P \to L$ разрезается горизонтально на две матрицы $P_1 \to L_1$ и $P_2 \to L_2$, так что первая матрица— это максимальный префикс исходной, такой, что правила для переменной, конструктора или or-паттернов применимо.

```
catch C(x; P1 -> L1) with C(x; P2 -> L2)
```

Оптимизации порождаемых let будут в других частях компилятора.

2.2.5 Оптимизируем mixture rule

Рассмотрим следующую функцию merge, которая обрабатывает два списка, и построим для нее матрицу $(P \to L)$.

```
let merge lx ly = match lx,ly with
| [], _ -> 1
| _, [] -> 2
| x::xs, y::ys -> 3
```

$$(P \to L) = \begin{bmatrix} [\,] & _ & \to 1 \\ _ & [\,] & \to 2 \\ x :: xs & y :: ys & \to 3 \end{bmatrix}$$

Применяем дважды mixture rule

$P_1 \to L_1 = ([\,]\ _ \to 1)$
$P_2 \to L_2 = (_\ [\,] \to 2)$
$P_3 \to L_3 = (x :: xs\ y :: ys \to 3)$

У нас получилось 3 матрицы, однако можно заметить, что можно поменять местами 2 и 3 строки, т.к. во втором столбце у них разные паттерны: во второй строке это пустой список, а в третьей - хотя бы с одним элементом. Тогда после перестановки мы получим:

$$(P \to L) = \begin{bmatrix} [\,] & _ & \to 1 \\ x :: xs & y :: ys & \to 3 \\ _ & [\,] & \to 2 \end{bmatrix}$$

Это позволяет нам разбить исходную матрицу уже на две и соответственно породить меньше кода, который будет работать быстрее.
Код для разрезания на 3 матрицы:

```
catch
    (catch
        (switch lx with case []: 1
            default: exit)
        with (catch
            (switch ly with case []: 2
                default: exit)
        with (catch
            (switch lx with
                case (::):
                    (switch ly with
                        case (::) : 3
                        default: exit)
```

```
                    default: exit))))
with (failwith "Partial match")
```

Код для разрезания на две матрицы:

```
catch
    (catch
        (switch* lx with
            case []: 1
            case (::) :
                (switch ly with
                    case (::): 3
                    default: exit))
    with
        (switch ly with
            case []: 2
            default: exit))
with (failwith "Partial match")
```

Этот пример можно еще оптимизировать, используя информацию о полноте. Нужно заметить, что данный PM, разбирает все возможные комбинации lx и ly, следовательно исключения ”Partial match” быть не может, достаточно заметить, что при первом matching ly, если ly не содержит хотя бы одного элемента, значит он пустой. Это позволяет нам избавиться полностью от второго matching ly, т.к. он избыточен.

```
catch
    (switch* lx with
        case []: 1
        case (::) :
            (switch ly with
                case (::): 3
                default: exit))
with 2
```

2.2.6 Сложный пример

Рассмотрим более сложный пример. Введем следующий тип списка, где есть отдельный конструктор для списка из одного элемента.

```
type t = Nil | One of int | Cons of int * t
```

Возьмем следующую матрицу PM.

$$(P \to L) = \begin{bmatrix} Nil & _ & \to & 1 \\ _ & Nil & \to & 2 \\ One(x) & _ & \to & 3 \\ _ & One(y) & \to & 4 \\ Cons(x, xs) & Cons(y, ys) & \to & 5 \end{bmatrix}$$

После применения приведенной выше оптимизации mixture rule мы получим 4 матрицы

$$(P \to L) = \begin{bmatrix} Nil & _ & \to & 1 \\ Cons(x, xs) & Cons(y, ys) & \to & 5 \end{bmatrix}$$
$$(P \to L) = \begin{bmatrix} _ & Nil & \to & 2 \end{bmatrix}$$
$$(P \to L) = \begin{bmatrix} One(x) & _ & \to & 3 \end{bmatrix}$$
$$(P \to L) = \begin{bmatrix} _ & One(y) & \to & 4 \end{bmatrix}$$

Код компиляции получится следующим.

```
catch
 (catch
  (catch
   (switch lx with
    case Nil: 1
    case Cons:
     (switch ly with
      case Cons: 5
      default: exit)
    default: exit)
   with
    (switch ly with
     case Nil: 2
     default: exit))
 with
  (switch lx with
   case One: 3
   default: exit))
with 4
```

```
1 catch
2  (catch
3   (catch
4    (switch lx with
5     case Nil: 1
6     case Cons:
7      (switch ly with
8       case Cons: 5
9       case Nil: (exit 2)
10      case One: (exit 4)
11     default: (exit 2))
12   with (2)
13    (switch ly with
14     case Nil: 2
15     default: (exit 3)))
16  with (3)
17  (switch lx with
18   case One: 3
19   default: (exit 4)))
20 with (4) 4
```

Мотивационный пример: $(Cons(..,..), One(..))$. Путь для кода слева 7-13-17-20 хотим превратить в 7-20, путем добавления меток для блоков exit и with, теперь exit будет возвращать не к внешнему with, а к with с той меткой, которая указана после exit. Что может сократить кол-во переходов и улучшить эффективность кода.

Рассмотрим более агрессивные оптимизации. В обработчик 3 можно попасть только из 2, в ветке, где ly != Nil. В свою очередь в обработчик 3 можно попасть из двух мест:

```
9 (lx ly) = (Cons (_, _) Nil)
11 lx = One v
```

Так как в строке 15 exit сработает только когда ly != Nil, можно сделать вывод, что первый случай не приведет к обработчику 3. Как правило, такие оптимизации можно провернуть позже, во время dead code elimination.

К цифрам для exit можно добавить ещё и выражения, которые будут соответствовать паттернам-переменным. В коде они превратятся просто в мутабельную память, куда будут присваивать. Так мы поддержим переменные в or-паттернах. Рассмотрим следующий пример:

```
let car list = match list with
| Nil -> -1
| (One x | Cons (x,_)) -> x
```

Компиляция состоит в выделении нового обработчика with под номером 2 и расширением клаузы $Onex|Cons(x,_)$ на две клаузы с шаблонами $Onex$ и $Cons(x,_)$. Действия для новых клаузул - это exit 2.

```
catch C((list), (
    Nil → -1
    One x' → (exit 2 x')
    Cons (x', _) → (exit 2 x')))
with (2 x) C((),( → x))
```

Обратите внимание, что и выходы, и обработчики ловушек теперь принимают еще один дополнительный аргумент: вхождения x’ в exit не являются обязательными и ссылаются на переменные шаблона, а вхождения x в обработчиках являются обязательными. Эта новая конструкция позволяет компилировать or-паттерны с переменными. Реализация не очень сложна: конструкция catch ... with (2 x) ... выделяет одну изменяемую переменную; выход обновляет эту переменную, которая считывается перед входом в обработчик. В компиляторе нативного кода такая переменная является временной и, в конечном счете, машинным регистром. Сгенерированный лямбда-код выглядит следующим образом:

```
catch
  switch* list with
    case Nil: -1
```

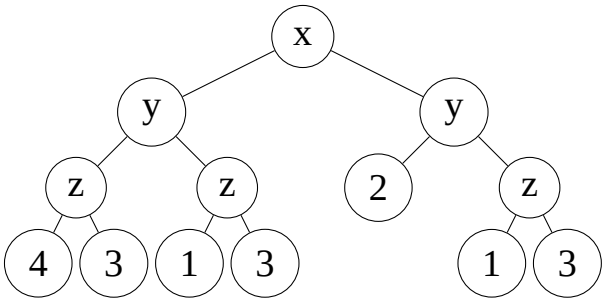
```
    case One: (exit 2 (field 0 list))
    case Cons: (exit 2 (field 0 list))
with (2 x) x
```

3 Диаграмма решений

В этой секции поговорим о принципиально другом подходе компиляции — диаграммы решений. Диаграмма решений — это дерево (DAG), где в листьях результаты, в узлах вычисляется некоторые выражения, а на ребрах написаны возможные его значения. Для нашего случая в листьях результат сопоставления, в узле подвыражение, а на ребрах названия конструкторов. Мы будем строить диаграмму решений, где на каждом из уровней мы будем сравнивать определенный паттерн. Это гарантирует нам, что мы не будем повторно проводить сравнение для паттернов. Рассмотрим следующий пример.

```
match x,y,z with
| _, F, T -> 1
| F, T, _ -> 2
| _, _, F -> 3
| _, _, T -> 4
```

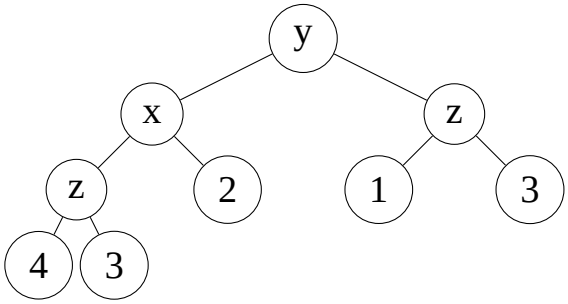
```
if x then
  if y then
    if z then 4 else 3
  else
    if z then 1 else 3
else
  if y then 2
  else
    if z then 1 else 3
```



Заметим, что у нас есть общие поддеревья для случая, когда y = F. Это можно использовать для следующий оптимизации: можно начать строить дерево с разбора не x, а y. Это позволит нам уменьшить наше дерево, его высота останется прежней, а количество сравнений для случая: x = _, y = F, z = _, сократится с двух до трёх.

```
match x,y,z with
| _, F, T -> 1
| F, T, _ -> 2
| _, _, F -> 3
| _, _, T -> 4
```

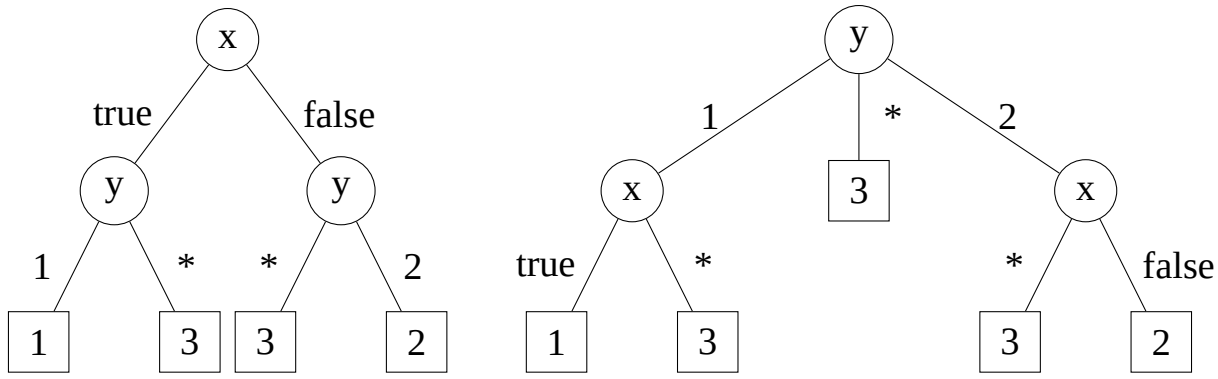
```
if x then
  if y then
    if z then 4 else 3
  else
    if z then 1 else 3
else
  if y then 2
  else
    if z then 1 else 3
```



Для обобщения этой оптимизации введем следующую эвристику (necessity) для каждой переменной (столбца): посчитаем кол-во возможных значений для нее — кол-во путей из паттерна. Рассмотрим следующий пример:

$$P = \begin{bmatrix} true & 1 & \rightarrow & ... \\ false & 2 & \rightarrow & ... \\ _ & _ & \rightarrow & ... \end{bmatrix} \quad N = \begin{bmatrix} \bullet & \bullet \\ \bullet & \bullet \\ & \bullet \end{bmatrix}$$

Для первого столбца возможно только два значения, так как третий вариант (wildcard) — избыточен, а для второго столбца возможных вариантов 3, поэтому именно с него следует начать построения дерева решений. В общем случае вычисление данной эвристики это очень непростая задача, иначе бы построение диаграммы решений было линейным, однако это не так, что сильно тормозит компиляцию таким образом. Для нашего примера неэффективная и эффективная диаграммы решений будут следующими:



Как можно заметить, для второго дерева существует случай y <> 1 и y <> 2, когда нам потребуется всего одна проверка вместо двух, как в не оптимизированном случае.

Стоит сказать, что диаграмма решений не предоставляет никакого преимущества для большинства программ на OCaml. Для того, чтобы проявилась эффективность построения диаграммы решений, необходимо достаточное разнообразие типов и паттернов, как в примере ниже (реализация интерпретации стекового языка[1]). По всем причинам выше в OCaml до сих пор используются автоматы с возвратами.

```
type acc =
    | Int of int
    | Clo of int * int
type code =
    | Ldi of int
    | Push
    | Val of acc
    | IOp of int
    | Test of int * int
    | Extend
    | Search of int
    | Pushenv
    | Popenv
    | Mkclos of int
    | Mkclosrec of int
    | Apply
    | Env of int
    | Code of int
```

[1] ссылка?

```
let rec run a s c =
    match (a, s, c) with
    | _, _, Ldi i :: c -> 1
    | _, _, Push :: c -> 2
    | Int n2, Val (Int _)::s, IOp _ :: c -> 3
    | Int 0, _, Test (c2, _) :: c -> 4
    | Int _, _, Test (_, c3) :: c -> 5
    | _, _, Extend :: c -> 6
    | _, _, Search k :: c -> 7
    | _, _, Pushenv :: c -> 8
    | _, Env e :: s, Popenv :: c -> 9
    | _, _, Mkclos cc :: c -> 10
    | _, _, Mkclosrec cc :: c -> 11
    | Clo (cc, ce), Val v :: s, Apply::c -> 12
    | a, Code c :: Env e :: s, [] -> 13
    | a, [], [] -> 14
```

4 Языки с ленивой семантикой (Haskell)

В ленивых языках есть понятие еще не посчитанного значения. В Haskell тип может быть наделен дополнительно расходящимся термом. При попытке его вычислить программа завершится с ошибкой. Пусть у нас есть функция or, заданная следующим образом:

```
(* Type of "or" *)
or :: Bool -> Bool -> Bool
(* Implementation *)
or True _ = True
or _ True = True
or False False = False
```

Если мы попытаемся вычислить её от True и undefined (расходящийся терм), то получим ожидаемое значение True. Но если в попытке оптимизировать pattern matching мы поменяем местами первую и вторую строку, то такой вызов упадет с ошибкой, так как в таком случае нам *необходимо вычислить второе значение*. В ленивых языках у нас нет свободы в выборе первого столбца, следовательно в строгих больше простора для оптимизаций.